Phase 2

Importy

In [57]:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
import os
import numpy as np
from pathlib import Path

from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler, PowerTransformer, QuantileTransformer

from sklearn.feature_selection import f_classif, mutual_info_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression


TARGET = "oximetry"

# naciatanie dat
DATA_DIR = "./115"

df_obs = pd.read_csv(os.path.join(DATA_DIR, "observation.csv"), sep='\t', encoding='utf-8-sig')
df_pat = pd.read_csv(os.path.join(DATA_DIR, "patient.csv"), sep='\t', encoding='utf-8-sig')
df_sta = pd.read_csv(os.path.join(DATA_DIR, "station.csv"), sep='\t', encoding='utf-8-sig')
```

2.1 Realizácia predspracovaina dát

2.1 (A) Predpriprava

Dame si vyprintovat tabulku observation aby sme mali rychly nahlad do toho s cim pracujeme

In [58]:

```python
def dataset_summary(df, name):
    print("Dataset:",name)
    print("Počet riadkov: ",df.shape[0])
    print("Počet stĺpcov: ",df.shape[1])
    display(df.head())

dataset_summary(df_obs, "observation.csv")
```

```
Dataset: observation.csv
Počet riadkov:  12169
Počet stĺpcov:  23
```

	SpO$_2$	HR	PI	RR	EtCO$_2$	FiO$_2$	PRV	BP	Skin Temperature	Motion/Activity index	...	CO	Blood Flow Index	PPG waveform features	Signal Quality Index	Respir
0	96.634591	80.009013	8.685891	17.518665	41.509058	45.163316	144.295418	104.803180	34.978370	10.828257	...	4.088809	40.790753	53.666253	62.281085	54.9
1	96.640621	74.589540	13.724679	15.588010	40.198284	50.791918	110.819791	103.974975	35.193140	9.818858	...	4.036563	43.207084	58.219436	45.805412	58.8
2	98.146757	85.019981	7.766128	18.747887	41.964458	63.858679	91.591179	106.573202	35.938901	9.767053	...	4.224814	48.594592	56.163501	45.538311	46.3
3	98.391300	73.823193	11.114940	15.905309	42.781757	55.433424	59.373133	103.438804	34.843787	12.695630	...	4.030798	51.561544	42.193516	44.665135	27.8
4	97.391221	78.702018	12.126423	14.923299	38.946213	52.511153	100.147921	102.652685	36.109030	10.813473	...	4.070682	51.560475	45.447019	63.837901	50.3

5 rows × 23 columns

V rýchlosti vidíme že všetky stĺpce sú typu float64, nemáme žiadne chýbajúce hodnoty.

Ďalej si rozdelíme dáta na vstupy, čize celá tabuľka bez oximetry, a výstupy, oximetry hodnoty. Vyhodili sme stĺpce latitude a longitude, ktoré nemajú s oximetry nič spoločné

In [59]:

```python
Y = df_obs["oximetry"]
X = df_obs.drop(columns=["oximetry"])
X = X.drop(columns=["latitude", "longitude"])

display(Y.head(10))
display(X.head(10))
```

```
0    1.0
1    0.0
2    1.0
3    1.0
4    1.0
5    0.0
6    0.0
7    1.0
8    1.0
9    0.0
Name: oximetry, dtype: float64
```

	SpO$_2$	HR	PI	RR	EtCO$_2$	FiO$_2$	PRV	BP	Skin Temperature	Motion/Activity index	PVI	Hb level	SV	CO	Bloo Flow Inde
0	96.634591	80.009013	8.685891	17.518665	41.509058	45.163316	144.295418	104.803180	34.978370	10.828257	14.037362	15.057155	90.483653	4.088809	40.79075
1	96.640621	74.589540	13.724679	15.588010	40.198284	50.791918	110.819791	103.974975	35.193140	9.818858	14.088573	14.188352	77.917910	4.036563	43.20708
2	98.146757	85.019981	7.766128	18.747887	41.964458	63.858679	91.591179	106.573202	35.938901	9.767053	15.574370	14.773033	89.239098	4.224814	48.59459
3	98.391300	73.823193	11.114940	15.905309	42.781757	55.433424	59.373133	103.438804	34.843787	12.695630	13.884048	15.141773	81.060225	4.030798	51.56154
4	97.391221	78.702018	12.126423	14.923299	38.946213	52.511153	100.147921	102.652685	36.109030	10.813473	14.657350	13.834966	75.759984	4.070682	51.56047
5	98.370074	86.754404	10.286109	17.971480	37.238983	56.432166	69.915445	101.622527	35.472269	9.986186	15.805165	13.513272	80.743617	4.315589	42.84987
6	98.713169	77.180663	9.763689	18.186233	36.955089	47.046158	70.942694	109.006225	35.937648	9.291926	13.170684	14.997614	79.732704	4.051470	24.13083
7	98.980978	85.181119	10.789893	16.435032	42.984438	54.918383	81.807949	103.353147	34.076874	10.466959	15.062680	14.569688	78.293959	4.236177	59.47152
8	96.433581	76.715451	10.164767	16.451191	42.341025	63.552583	163.976927	108.954107	35.356968	10.615501	14.630771	14.775714	82.915962	4.046473	45.76492
9	97.867503	71.841157	10.839820	14.017102	39.056205	45.204822	93.668655	100.294404	35.994043	10.060609	15.105803	15.546249	85.481459	4.023458	53.84442

Ďalej si rozdelíme dáta na trénovaciu a testovaciu množinu. Použili sme pomer 80 ku 20 čož by pri velkosť nášho datasetu malo byť ideálne

In [60]:

```python
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y,
    test_size=0.2,
    stratify=Y,
    random_state=42
)

# Overenie výsledkov
print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
print("\nPomer tried v train:")
print(Y_train.value_counts(normalize=True))
print("\nPomer tried v test:")
print(Y_test.value_counts(normalize=True))
```

```
Train shape: (9735, 20)
Test shape: (2434, 20)

Pomer tried v train:
oximetry
1.0    0.596199
0.0    0.403801
Name: proportion, dtype: float64

Pomer tried v test:
oximetry
1.0    0.596138
0.0    0.403862
Name: proportion, dtype: float64
```

2.1 (B) Príprava testovacej množiny

1. Skontrolujeme či všetky atribúty sú numerické
2. Odstránime duplikáty aby sme zabezpečili že máme jedeno meranie jeden riadok
3. Nahradíme nekonečné hodnoty a chýbajúce hodnoty

Používame imputer, ktorý sa naučí na trénovacej množine pomocou fit a hneď aj aplikuje pomocou transform. Toto sa spraví naraz pomocou funkcie `fit_transform()`. Následne na testovacích dátach použijeme už iba transform aby sme aplikovali naučené dáta na testovacej množine a neporušili integritu.

In [61]:

```python
# kontrola duplikatov
non_numeric = [c for c in X.columns if not np.issubdtype(X[c].dtype, np.number)]
assert not non_numeric, f"nie sú numerické: {non_numeric}"

# odstranenie duplikatov
dup_mask = X_train.duplicated(keep="first")
n_dups = int(dup_mask.sum())
if n_dups:
    keep_idx = ~dup_mask
    X_train = X_train.loc[keep_idx]
    Y_train = Y_train.loc[keep_idx]

# nahradenie nekonecnych a chybajucich hodnot
X_train = X_train.replace([np.inf, -np.inf], np.nan)

# imputer
imputer = SimpleImputer(strategy="median")
X_train_clean_arr = imputer.fit_transform(X_train)
X_train_clean = pd.DataFrame(X_train_clean_arr, index=X_train.index, columns=X_train.columns)
```

Takto pripavené dáta si uložíme do csv

In [62]:

```python
X_train_clean.to_csv("phase2_train_clean.csv", index=True)
Y_train.to_csv("phase2_y_train.csv", index=True)
```

Spravíme to isté pre testovaciu množinu

In [63]:

```python
# Nahradiť ±inf -> NaN
X_test_fixed = X_test.replace([np.inf, -np.inf], np.nan)

# Transform
X_test_clean_arr = imputer.transform(X_test_fixed)
X_test_clean = pd.DataFrame(X_test_clean_arr, index=X_test.index, columns=X_test.columns)

# Ulozime
X_test_clean.to_csv("phase2_test_clean.csv", index=True)
Y_test.to_csv("phase2_y_test.csv", index=True)
```

2.1 (C) Transformácia dát

Následne sme transformovali dáta, ktoré neskôr budeme používať na trénovanie. Vytvorili sme 4 rôzne varianty datasetov pomocou rôznych spôsobov scalingu a transformovania

Metódy na škálovanie:

1. Standard Scaler Každá premenná má priemer približne 0 a smerodajku približne 1
2. MinMax Scaler Prevádza hodnoty do intervalu `<0,1>`

Metódy na trasformovanie:

1. PowerTransformer (Yeo–Johnson) + StandardScaler Upravuje tvar rozdelenie, zmenšuje šikmosť, stabilizuje varianciu, potom normalizuje rozsah.
2. QuantileTransformer - normal + RobustScaler Rozloženie premieňa na približne normálne a robustne škáluje, čo potláča vplyv outlinerov

Uloženie dát

Výsledky všetkých metód sa spoja (X+Y) a uložia ako samostatné datasety

Načítanie

In [64]:

```python
DATA_DIR = "./"
OUT_DIR = "./processed"
Path(OUT_DIR).mkdir(parents=True, exist_ok=True)

# načítaj čisté výstupy
X_train_clean = pd.read_csv(Path(DATA_DIR) / "phase2_train_clean.csv", index_col=0)
X_test_clean  = pd.read_csv(Path(DATA_DIR) / "phase2_test_clean.csv",  index_col=0)

# y ako Series
Y_train = pd.read_csv(Path(DATA_DIR) / "phase2_y_train.csv", index_col=0).squeeze("columns")
Y_test  = pd.read_csv(Path(DATA_DIR) / "phase2_y_test.csv",  index_col=0).squeeze("columns")

print("Krok 0 OK:", X_train_clean.shape, X_test_clean.shape, "target =", TARGET)
```

```
Krok 0 OK: (9623, 20) (2434, 20) target = oximetry
```

Transformácie

In [65]:

```python
# Poistky: y nesmie byť medzi features
assert "oximetry" not in X_train_clean.columns, "oximetry je vo features"
assert set(X_train_clean.columns) == set(X_test_clean.columns)
feature_cols = list(X_train_clean.columns)
TARGET = "oximetry"
target_name = TARGET

OUT_DIR = Path(OUT_DIR)
OUT_DIR.mkdir(parents=True, exist_ok=True)

def save_variant(name, Xtr_df, Xte_df, Ytr, Yte):
    # sanity checks
    assert np.isfinite(Xtr_df.to_numpy()).all(), f"{name}: NaN/inf v train"
    assert np.isfinite(Xte_df.to_numpy()).all(), f"{name}: NaN/inf v test"
    # pripojenie targetu
    tr = Xtr_df.copy(); tr[target_name] = Ytr
    te = Xte_df.copy(); te[target_name] = Yte
    # uloženie
    tr_path = OUT_DIR / f"train_{name}.csv"
    te_path = OUT_DIR / f"test_{name}.csv"
    tr.to_csv(tr_path, index=False); te.to_csv(te_path, index=False)
    print(f"[OK] {name} → {tr_path.name} | {te_path.name}  (shapes: {tr.shape}, {te.shape})")

# -------- Scaling 1: StandardScaler --------
sc_std = StandardScaler()
Xtr_std = pd.DataFrame(sc_std.fit_transform(X_train_clean), index=X_train_clean.index, columns=feature_cols)
Xte_std = pd.DataFrame(sc_std.transform(X_test_clean),      index=X_test_clean.index,  columns=feature_cols)
save_variant("std", Xtr_std, Xte_std, Y_train, Y_test)

# -------- Scaling 2: MinMaxScaler --------
sc_minmax = MinMaxScaler()
Xtr_minmax = pd.DataFrame(sc_minmax.fit_transform(X_train_clean), index=X_train_clean.index, columns=feature_cols)
Xte_minmax = pd.DataFrame(sc_minmax.transform(X_test_clean),      index=X_test_clean.index,  columns=feature_cols)
save_variant("minmax", Xtr_minmax, Xte_minmax, Y_train, Y_test)

# -------- Transformer 1: Power(Yeo-Johnson) + StandardScaler --------
pt = PowerTransformer(method="yeo-johnson")         # fit iba na train
Xtr_pt = pd.DataFrame(pt.fit_transform(X_train_clean), index=X_train_clean.index, columns=feature_cols)
Xte_pt = pd.DataFrame(pt.transform(X_test_clean),      index=X_test_clean.index,  columns=feature_cols)

sc_after_pt = StandardScaler()
Xtr_power = pd.DataFrame(sc_after_pt.fit_transform(Xtr_pt), index=Xtr_pt.index, columns=feature_cols)
Xte_power = pd.DataFrame(sc_after_pt.transform(Xte_pt),     index=Xte_pt.index, columns=feature_cols)
save_variant("power", Xtr_power, Xte_power, Y_train, Y_test)

# -------- Transformer 2: Quantile(→ normal) + RobustScaler --------
qt = QuantileTransformer(output_distribution="normal",
                         n_quantiles=min(1000, len(X_train_clean)),
                         random_state=42)
Xtr_q = pd.DataFrame(qt.fit_transform(X_train_clean), index=X_train_clean.index, columns=feature_cols)
Xte_q = pd.DataFrame(qt.transform(X_test_clean),      index=X_test_clean.index,  columns=feature_cols)

sc_robust = RobustScaler()
Xtr_quant = pd.DataFrame(sc_robust.fit_transform(Xtr_q), index=Xtr_q.index, columns=feature_cols)
Xte_quant = pd.DataFrame(sc_robust.transform(Xte_q),     index=Xte_q.index, columns=feature_cols)
save_variant("quantile", Xtr_quant, Xte_quant, Y_train, Y_test)

print("\nHotovo: 4 varianty uložené v", OUT_DIR.resolve())
```

```
[OK] std → train_std.csv | test_std.csv  (shapes: (9623, 21), (2434, 21))
[OK] minmax → train_minmax.csv | test_minmax.csv  (shapes: (9623, 21), (2434, 21))
[OK] power → train_power.csv | test_power.csv  (shapes: (9623, 21), (2434, 21))
[OK] quantile → train_quantile.csv | test_quantile.csv  (shapes: (9623, 21), (2434, 21))

Hotovo: 4 varianty uložené v /Users/nikolas/school/IAU/IAU_project/processed
```

Kontrola

In [66]:

```python
DIR = Path("./processed")
VARIANTS = ["std", "minmax", "power", "quantile"]

def check_variant(name):
    train_p = DIR / f"train_{name}.csv"
    test_p  = DIR / f"test_{name}.csv"
    df_tr = pd.read_csv(train_p)
    df_te = pd.read_csv(test_p)

    # Rovnaké feature stĺpce (bez targetu) a rovnaké poradie
    feats_tr = list(df_tr.columns[:-1])
    feats_te = list(df_te.columns[:-1])
    assert feats_tr == feats_te, f"{name}: train/test majú iné stĺpce alebo iné poradie"

    # Bez NaN/inf vo features
    Xtr = df_tr.iloc[:, :-1]
    Xte = df_te.iloc[:, :-1]
    assert np.isfinite(Xtr.to_numpy()).all(), f"{name}: NaN/inf v train features"
    assert np.isfinite(Xte.to_numpy()).all(), f"{name}: NaN/inf v test features"

    # Target má len hodnoty 0/1
    ytr = df_tr["oximetry"].values
    yte = df_te["oximetry"].values
    assert set(np.unique(ytr)).issubset({0,1}), f"{name}: train y nie je binárny"
    assert set(np.unique(yte)).issubset({0,1}), f"{name}: test y nie je binárny"

    # Pomer tried podobný v train/test
    r_tr = ytr.mean()
    r_te = yte.mean()
    diff = abs(r_tr - r_te)
    # tolerancia 1 %
    assert diff <= 0.01, f"{name}: rozdiel podielu tried train vs test > 1% (train={r_tr:.4f}, test={r_te:.4f})"

    # 6) Očakávané štatistiky podľa techniky (rýchla sanity)
    desc = Xtr.describe().T[["mean","std","min","max"]]

    if name == "std":
        # priemer ~0, std ~1
        m = desc["mean"].abs().median()
        s = desc["std"].median()
        print(f"[{name}] mean={m:.3f}, std={s:.3f}  (očakávanie: mean=0, std=1)")
    elif name == "minmax":
        # väčšina hodnôt v (0,1)
        min_ok = (desc["min"] >= -1e-9).mean()
        max_ok = (desc["max"] <= 1+1e-9).mean()
        print(f"[{name}] % stĺpcov s min>=0: {100*min_ok:.1f}%, max<=1: {100*max_ok:.1f}%  (očakávanie =100%)")
    elif name == "power":
        m = desc["mean"].abs().median()
        s = desc["std"].median()
        print(f"[{name}] mean={m:.3f}, std={s:.3f}  (očakávanie: mean=0, std=1)")
    elif name == "quantile":
        # po Quantile→normal + Robust: medián ~0, IQR = 1 (robustný scale)
        q1 = Xtr.quantile(0.25)
        q3 = Xtr.quantile(0.75)
        iqr = (q3 - q1).median()
        med = Xtr.median().abs().median()
        print(f"[{name}] median={med:.3f}, IQR={iqr:.3f}  (očakávanie: median=0, IQR=1)")

    print(f"[{name}] OK - shapes: train {df_tr.shape}, test {df_te.shape}\n")

for v in VARIANTS:
    check_variant(v)

print("Všetky kontroly prebehli.")
```

```
[std] mean=0.000, std=1.000  (očakávanie: mean=0, std=1)
[std] OK - shapes: train (9623, 21), test (2434, 21)

[minmax] % stĺpcov s min>=0: 100.0%, max<=1: 100.0%  (očakávanie =100%)
[minmax] OK - shapes: train (9623, 21), test (2434, 21)

[power] mean=0.000, std=1.000  (očakávanie: mean=0, std=1)
[power] OK - shapes: train (9623, 21), test (2434, 21)

[quantile] median=0.000, IQR=1.000  (očakávanie: median=0, IQR=1)
[quantile] OK - shapes: train (9623, 21), test (2434, 21)

Všetky kontroly prebehli.
```

2.1 (D)

V tejto fáze sme pripravili dáta na trénovanie modelov strojového učenia. Naším cieľom bolo získať čistý, číselný a konzistentný dataset, ktorý model dokáže spracovať bez chýb a skreslení.

Najskôr sme dáta rozdelili na trénovaciu a testovaciu množinu, aby sme mohli objektívne hodnotiť výkon modelov a zabránili tomu, že by sa model učil aj z testovacích dát. Následne sme dataset vyčistili – odstránili sme duplikáty, nahradili chýbajúce a nekonečné hodnoty mediánmi a ponechali sme iba numerické atribúty, keďže knižnica scikit-learn pracuje len s číselnými vstupmi.

Po vyčistení sme aplikovali štyri rôzne techniky transformácie dát. Použili sme dva scalery a dva transformery, aby sme neskôr vedeli porovnať ich vplyv na výkon modelov:

- StandardScaler sme zvolili preto, že vycentruje dáta okolo nuly a nastaví jednotkovú smerodajnú odchýlku, čo pomáha modelom, ktoré predpokladajú normálne rozdelenie dát (napr. logistická regresia alebo SVM).

- MinMaxScaler prevádza hodnoty do intervalu ⟨0, 1⟩, čo je vhodné pre algoritmy citlivé na mierku, ako sú neurónové siete alebo k-najbližších susedov (k-NN).

- PowerTransformer (Yeo–Johnson) sme použili na úpravu tvaru rozdelenia atribútov tak, aby sa viac podobali normálnemu rozdeleniu a odstránila sa šikmosť, čo zlepšuje stabilitu modelov a znižuje vplyv extrémnych hodnôt.

- QuantileTransformer mapuje dáta podľa kvantilov na normálne rozdelenie a v kombinácii s RobustScalerom znižuje vplyv outlierov – teda extrémnych meraní, ktoré by mohli skresliť výsledky.

Každá transformácia bola fitnutá len na trénovacích dátach a následne aplikovaná na testovacích, aby sme zabránili úniku informácií medzi množinami (data leakage).

Výsledkom sú štyri varianty datasetov (standard, minmax, power, quantile), ktoré sú pripravené na trénovanie a porovnávanie modelov v ďalšej fáze. Tieto varianty nám umožnia zistiť, aký spôsob predspracovania najlepšie zlepšuje výkon modelov pre naše dáta.

2.2 Výber atribútov pre strojové učenie

2.2 (A),(B)

Načítali si dáta s jednotlivých variant (Metód vytvorených v prechádzajúcom kroku). Ale netestujeme s každou. Zvolili sme si na to variant std

In [67]:

```python
VARIANTS = ["std", "minmax", "power", "quantile"]  # ak máš iné mená, uprav

def load_variant(name: str) -> tuple[pd.DataFrame, pd.Series]:
    # nacita train csv
    df = pd.read_csv("./processed/" f"train_{name}.csv")
    X = df.iloc[:, :-1].copy()
    y = df["oximetry"].astype(int).copy()
    return X, y

# primarny variant = std
PRIMARY = "std"
X_train, y_train = load_variant(PRIMARY)
feature_names = X_train.columns.tolist()
X_train.shape, y_train.shape
```

Out[67]:

```
((9623, 20), (9623,))
```

Ďalej sme zistili, ktoré atribúty sú informatívne k predikovanej premennej. Použili sme tieto štyri techniky:

- **ANOVA F-test (f_classif)** – zisťuje, či sa priemerné hodnoty atribútu líšia medzi triedami (oximetry).
- **Mutual Information (MI)** – meria, ako veľmi daný atribút znižuje neistotu o cieľovej premennej (zachytí aj nelineárne vzťahy).
- **Random Forest (RF)** – určí, ktoré atribúty najviac prispievajú k presnosti stromového modelu.
- **Logistic Regression (LR)** – hodnotí dôležitosť atribútov na základe absolútnych hodnôt koeficientov modelu.

In [68]:

```python
def rank_series(name, values, feats):
    s = pd.Series(values, index=feats, name=name).astype(float)
    # normalizácia do 0..1 pre porovnávanie naprieč metódami
    rng = s.max() - s.min()
    s_norm = (s - s.min()) / (rng if rng > 0 else 1.0)
    s_norm.name = name
    return s, s_norm

# 1 ANOVA F-test
F_vals, p_vals = f_classif(X_train, y_train)
anova_raw, anova = rank_series("anova_F", F_vals, feature_names)

# 2 Mutual Information
mi_vals = mutual_info_classif(X_train, y_train, random_state=42)
mi_raw, mi = rank_series("mutual_info", mi_vals, feature_names)

# 3 RandomForest importance
rf = RandomForestClassifier(n_estimators=400, max_depth=None, random_state=42, n_jobs=-1, class_weight=None)
rf.fit(X_train, y_train)
rf_raw, rf_imp = rank_series("rf_importance", rf.feature_importances_, feature_names)

# 4 LogisticRegression koeficienty (absolútna hodnota)
lr = LogisticRegression(penalty="l2", solver="liblinear", random_state=42, max_iter=2000)
lr.fit(X_train, y_train)
lr_abs = np.abs(lr.coef_[0])
lr_raw, lr_imp = rank_series("logreg_coef_abs", lr_abs, feature_names)

# Spojíme do jednej tabuľky (normalizované skóre)
scores_norm = pd.concat([anova, mi, rf_imp, lr_imp], axis=1)
scores_raw  = pd.concat([anova_raw, mi_raw, rf_raw, lr_raw], axis=1)

# Ensemble skóre = priemer normalizovaných skóre
scores_norm["ensemble_mean"] = scores_norm.mean(axis=1)

# urobíme iba jednu tabuľku
def make_table(scores_norm: pd.DataFrame, top_n=10) -> pd.DataFrame:
    # ranky (1 = najlepšie)
    ranks = scores_norm.rank(ascending=False, method="min")
    ranks.columns = [c + "_rank" for c in ranks.columns]

    tbl = pd.concat([scores_norm, ranks], axis=1)
    tbl = tbl.sort_values("ensemble_mean", ascending=False)

    keep = [
        "ensemble_mean","anova_F","anova_F_rank", "mutual_info","mutual_info_rank", "rf_importance","rf_importance_rank", "logreg_coef_abs",
          "logreg_coef_abs_rank"
    ]
    return tbl[keep].head(top_n)

compact_top10 = make_table(scores_norm, top_n=10)
compact_top10
```

Out[68]:

	ensemble_mean	anova_F	anova_F_rank	mutual_info	mutual_info_rank	rf_importance	rf_importance_rank	logreg_coef_abs	logreg_coef_abs_rank
EtCO$_2$	1.000000	1.000000	1.0	1.000000	1.0	1.000000	1.0	1.000000	1.0
PRV	0.423871	0.194131	2.0	0.535485	2.0	0.454025	2.0	0.511843	2.0
RR	0.220686	0.091577	3.0	0.207063	3.0	0.207095	3.0	0.377010	3.0
PI	0.093890	0.016948	4.0	0.062080	5.0	0.113814	5.0	0.182720	4.0
SpO$_2$	0.065892	0.009873	5.0	0.083959	4.0	0.152884	4.0	0.016854	10.0
PVI	0.035336	0.000634	6.0	0.026909	7.0	0.066532	6.0	0.047269	5.0
Blood Flow Index	0.012254	0.000314	7.0	0.033508	6.0	0.004263	18.0	0.010930	13.0
Respiratory effort	0.009101	0.000015	19.0	0.019839	9.0	0.007484	8.0	0.009065	16.0
Signal Quality Index	0.008857	0.000044	14.0	0.005784	11.0	0.007230	9.0	0.022370	7.0
Skin Temperature	0.008211	0.000204	8.0	0.002439	12.0	0.005133	16.0	0.025069	6.0

2.2 (C) Zdokumentovanie

Pri výbere atribútov sme sa zamerali na kombináciu viacerých metŕod, ktoré spolu pokrývajú rôzne aspekty vzťahu medzi vstupnými atribútmi a cieľovou premennou *oximetry*. Cieľom bolo získať nielen lineárny pohľad na dáta, ale aj zachytiť možné nelineárne a interaktívne väzby medzi premennými.

ANOVA F-test sme použili, pretože ide o metódu, ktorá umožňuje rýchlo overiť, či sa priemerné hodnoty atribútov významne líšia medzi triedami

Mutual information (MI) dopĺňa ANOVA o schopnosť zachytiť nelineárne závislosti.

Random Forest sme použili ako metódu, pretože meria dôležitosť atribútov na základe ich vplyvu na presnosť predikcie v rozhodovacích stromoch. Tak isto je robustná voči šumu

Logistic Regression je lineárny model so zreteľnými koeficientami, ktoré sa dajú priamo interpretovať ako váhu jednotlivých atribútov.

Každá metóda teda skúma atribúty z iného uhla pohľadu. Všetky skóre sme znormalizovali na interval `<0,1>` a skombinovali do jedného ukazovateľa *ensemble_mean*. Ten vyjadruje priemernú dôležitosť atribútov naprieč všetkými prístupmi. Takýmto štýlom sme identifikovali a zoradili 10 atribútov, ktoré by mohli slúžiť na predikciu predikovanej premennej.

2.3 Replikovateľnosť predspracovania

2.3 (A),(B) Úprava testovacej množiny s použitím pipeline

Vyššie pri riešení úlohy 2.1 sme sa pomýlili a aplikovali sme niektoré úpravy aj na testovaciu množinu.

V ďaľšom kroku upravíme kód z úlohy 2.1 tak aby sa úpravy aplikovali aj na trénovaciu a zároveň aj na testovaciu bez narušenia procesu učenia. Čiže fit sa bude robť

In [69]:

```python
TARGET = "oximtetry"

# naciatanie dat
DATA_DIR = "./115"

df_obs = pd.read_csv(os.path.join(DATA_DIR, "observation.csv"), sep='\t', encoding='utf-8-sig')
df_pat = pd.read_csv(os.path.join(DATA_DIR, "patient.csv"), sep='\t', encoding='utf-8-sig')
df_sta = pd.read_csv(os.path.join(DATA_DIR, "station.csv"), sep='\t', encoding='utf-8-sig')

Y = df_obs["oximetry"]
X = df_obs.drop(columns=["oximetry"])

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y,
    test_size=0.2,
    stratify=Y,
    random_state=42
)

# Overenie výsledkov
print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
print("\nPomer tried v train:")
print(Y_train.value_counts(normalize=True))
print("\nPomer tried v test:")
print(Y_test.value_counts(normalize=True))

# kontrola duplikatov
non_numeric = [c for c in X.columns if not np.issubdtype(X[c].dtype, np.number)]
assert not non_numeric, f"Nasledujúce stĺpce nie sú numerické: {non_numeric}"

# odstranenie duplikatov
dup_mask = X_train.duplicated(keep="first")
n_dups = int(dup_mask.sum())
if n_dups:
    keep_idx = ~dup_mask
    X_train = X_train.loc[keep_idx]
    Y_train = Y_train.loc[keep_idx]

# nahradenie nekonecnych a chybajucich hodnot
X_train = X_train.replace([np.inf, -np.inf], np.nan)
```

```
Train shape: (9735, 22)
Test shape: (2434, 22)

Pomer tried v train:
oximetry
1.0    0.596199
0.0    0.403801
Name: proportion, dtype: float64

Pomer tried v test:
oximetry
1.0    0.596138
0.0    0.403862
Name: proportion, dtype: float64
```

In [70]:

```python
# replikovateľné predspracovanie s sklearn.pipeline
from pathlib import Path
import joblib
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler, PowerTransformer, QuantileTransformer

TARGET = "oximetry"
OUT_DIR = Path("./processed")
OUT_DIR.mkdir(parents=True, exist_ok=True)

#  Poradie stĺpcov musi byť totožné
assert list(X_train.columns) == list(X_test.columns), "Train/Test nemajú rovnaké stĺpce v rovnakom poradí."
feature_cols = list(X_train.columns)

def save_variant(name, Xtr, Xte, Ytr, Yte):
    # prevedieme na ndarray kvôli jednoznačnej práci s np.isfinite
    Xtr_np = np.asarray(Xtr)
    Xte_np = np.asarray(Xte)

    # bez NaN/inf
    assert np.isfinite(Xtr_np).all(), f"{name}: NaN/inf v train"
    assert np.isfinite(Xte_np).all(), f"{name}: NaN/inf v test"

    # sedí počet stĺpcov
    assert Xtr_np.shape[1] == len(feature_cols), f"{name}: Xtr má {Xtr_np.shape[1]} =! {len(feature_cols)} stĺpcov"
    assert Xte_np.shape[1] == len(feature_cols), f"{name}: Xte má {Xte_np.shape[1]} =! {len(feature_cols)} stĺpcov"

    # na DataFrame s rovnakými indexami ako y
    if not isinstance(Ytr, pd.Series):
        Ytr = pd.Series(Ytr)
    if not isinstance(Yte, pd.Series):
        Yte = pd.Series(Yte)

    Xtr_df = pd.DataFrame(Xtr_np, index=Ytr.index, columns=feature_cols)
    Xte_df = pd.DataFrame(Xte_np, index=Yte.index, columns=feature_cols)

    tr = Xtr_df.copy(); tr[TARGET] = Ytr.values
    te = Xte_df.copy(); te[TARGET] = Yte.values

    tr_path = OUT_DIR / f"train_{name}.csv"
    te_path = OUT_DIR / f"test_{name}.csv"
    tr.to_csv(tr_path, index=False)
    te.to_csv(te_path, index=False)

    print(f"[OK] {name} → {tr_path.name} | {te_path.name}  "
          f"(train: {tr.shape}, test: {te.shape})")

# 4 varianty ako Pipeliny
nq = min(1000, len(X_train))  # pre QuantileTransformer
pipelines = {
    "std": Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]),
    "minmax": Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", MinMaxScaler()),
    ]),
    "power": Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("power", PowerTransformer(method="yeo-johnson", standardize=False)),
        ("scale", StandardScaler()),
    ]),
    "quantile": Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("qt", QuantileTransformer(output_distribution="normal", n_quantiles=nq, random_state=42)),
        ("rb", RobustScaler(with_centering=True, with_scaling=True)),
    ]),
}

# Fit na TRAIN, transform na TEST, uloženie CSV a samotnej pipeline (replikovateľnosť)
for name, pipe in pipelines.items():
    Xtr = pipe.fit_transform(X_train, Y_train)
    Xte = pipe.transform(X_test)
    save_variant(name, Xtr, Xte, Y_train, Y_test)
    joblib.dump(pipe, OUT_DIR / f"preproc_{name}.joblib")

print("\nHotovo: 4 pipeline varianty uložené v", OUT_DIR.resolve())
```

```
[OK] std → train_std.csv | test_std.csv  (train: (9734, 23), test: (2434, 23))
[OK] minmax → train_minmax.csv | test_minmax.csv  (train: (9734, 23), test: (2434, 23))
[OK] power → train_power.csv | test_power.csv  (train: (9734, 23), test: (2434, 23))
[OK] quantile → train_quantile.csv | test_quantile.csv  (train: (9734, 23), test: (2434, 23))

Hotovo: 4 pipeline varianty uložené v /Users/nikolas/school/IAU/IAU_project/processed
```

Záver

Použitím Pipeline sme zabezpečili, že všetky kroky prespracovanai sa vykonávajú konzsitentne na trénovacej aj testovacej množine. Pipeline bola fitnutá iba na trénovacích dátach, čím by sa malo predísť data leakagu. Vyššiu uvedený kód zaručuje opakovateľnosť, prenositeľnosť a reprodukovateľnosť výsledkov aj pri spracovaní nových dát.

Podiela práce

Nikolas Knapik	Adam Svitek
60	40